Session 11: CI/CD Pipeline

DevOps Fundamentals

Rikkei Academy®
where the dream begins

© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

NỘI DUNG

Lesson 01: Khái niệm cốt lõi của CI, CD

Lesson 02: Thiết kế quy trình CI tự động

Lesson 03: Thiết kế quy trình CD tự động

Lesson 04: Viết file Pipeline hoàn chỉnh

© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

RIKEI Academy®
where the dream begins

1 Khái niệm cốt lõi của CI, CD

Quy trình phát hành thủ công

Nhật ký một lần phát hành thật, ghi lại theo mốc thời gian

```
RELEASE LOG — THỨ SÁU

[17:30] git pull trên máy cá nhân
[17:34] ./gradlew build -x test --no-daemon  # bỏ test cho nhanh
[17:41] scp build/libs/app.jar deploy@server:/tmp
[17:48] ssh user@server "systemctl stop app"
[17:49] ssh user@server "cp /tmp/app.jar /var/www/ && sudo systemctl start app"
[17:52] mở trình duyệt kiểm tra bằng mắt

[18:10] khách hàng báo lỗi HTTP 500
[18:11] "trên máy em vẫn chạy bình thường"
[18:40] rollback thất bại: không còn bản .jar cũ

# tổng thời gian: 70 phút, kết quả: sự cố
```

Không thể lặp lại

Mỗi người thực hiện một kiểu; kết quả phụ thuộc vào người bấm lệnh

Không có bước kiểm tra bắt buộc

Bỏ qua test khi gấp là điều luôn xảy ra, không ai ngăn được

Không quay lại được trạng thái cũ

Không có artifact đánh phiên bản nào để rollback khi sự cố

Mục tiêu của CI/CD: chuyển quy trình phụ thuộc con người thành một kịch bản được phiên bản hoá, chạy giống nhau ở mọi lần phát hành.

© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

Phạm vi của CI, CD

Ba khái niệm cùng nằm trên một chuỗi công việc, khác nhau ở điểm dừng tự động hoá



Continuous Integration	Tự động build và test mọi commit
Continuous Delivery	Sản phẩm luôn sẵn sàng phát hành, lên production do người quyết định
Continuous Deployment	Tự động tới production khi mọi bước kiểm thử đều đạt

Độ dài mỗi dải chính là phạm vi được tự động hoá

CI trả lời: mã nguồn có tích hợp được không?
Phát hiện lỗi trong vài phút thay vì vài tuần, ngay khi commit vừa được đẩy lên

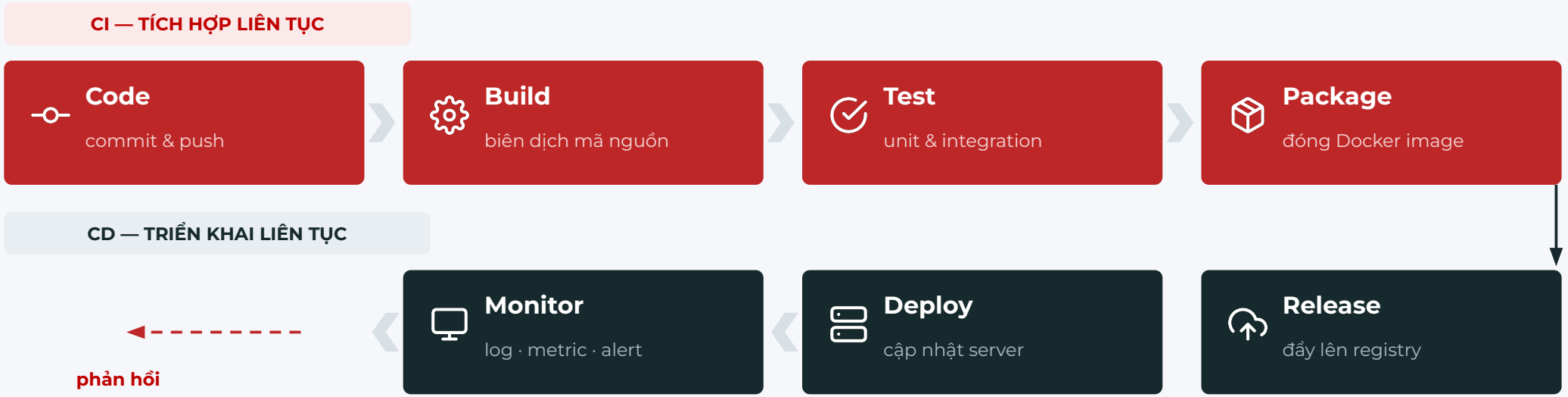
Delivery trả lời: đã sẵn sàng phát hành chưa?
Artifact đã qua kiểm thử và nằm chờ; con người chọn thời điểm đưa lên production

Deployment trả lời: đã lên production chưa?
Không còn thao tác thủ công nào; mỗi commit đạt chuẩn là một lần phát hành

© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

Vòng lặp CI/CD

Bảy chặng nối thành một vòng khép kín; kết quả giám sát quay lại thành yêu cầu mới



Mỗi chặng đều tự động và để lại log
Khi pipeline đỏ, log chỉ đúng chặng nào thất bại và vì sao

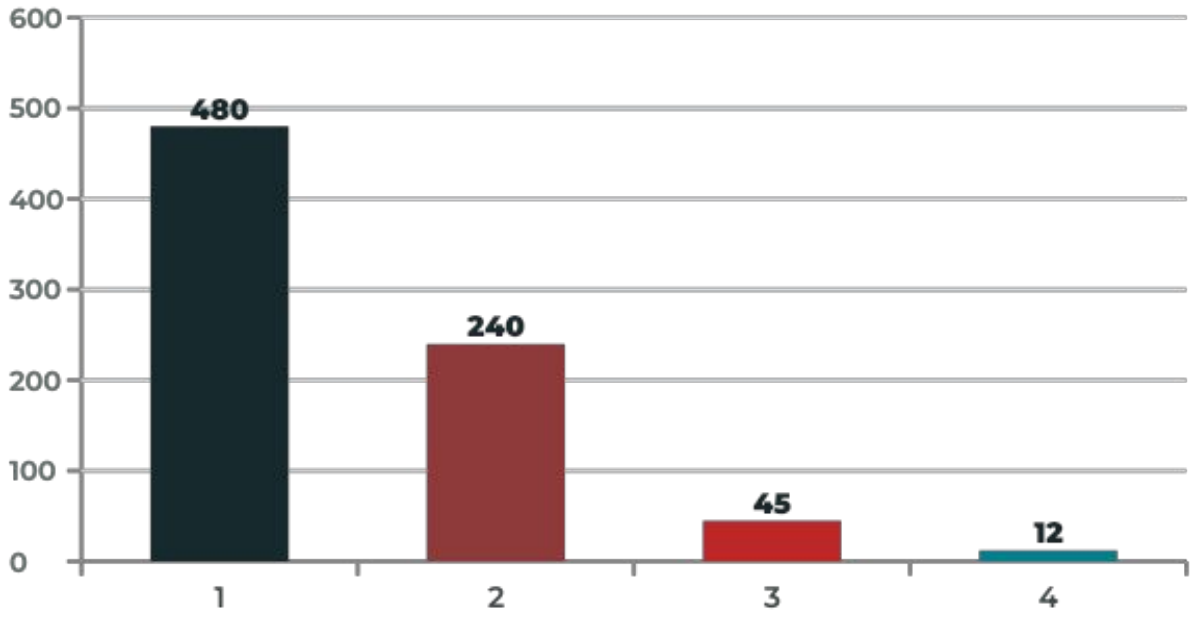
Image được build đúng một lần
Server chỉ kéo về image đã qua kiểm thử, không biên dịch lại từ mã nguồn

Giám sát là đầu vào của vòng sau
Số liệu lỗi và độ trễ quyết định ưu tiên cho lần phát hành tới

© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

Chỉ số đo lường hiệu quả pipeline

Thời gian từ khi commit được đẩy lên đến khi thay đổi có mặt trên production, đơn vị phút



Số liệu tham khảo cho một dịch vụ backend cỡ trung; giá trị tuyệt đối khác nhau theo dự án.

BỐN CHỈ SỐ CẦN THEO DÕI

1 Tần suất phát hành

Số lần đưa thay đổi lên production trong một khoảng thời gian

2 Lead time

Khoảng thời gian từ commit đến khi chạy trên production

3 Tỷ lệ phát hành lỗi

Phần trăm lần phát hành phải sửa gấp hoặc rollback

4 Thời gian khôi phục

Khoảng thời gian từ lúc phát hiện sự cố đến khi khắc phục xong sự cố

Đo trước khi tối ưu: không có số liệu thì mọi cải tiến pipeline chỉ là cảm nhận.

© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

So sánh ba công cụ CI/CD hàng đầu

Cú pháp khác nhau nhưng mô hình giống nhau: sự kiện, job, step và runner

Tiêu chí	GitHub Actions	Jenkins	GitLab CI
Mô hình triển khai	SaaS, không cần cài đặt	Tự cài và tự vận hành	SaaS hoặc self-managed
File cấu hình	YAML trong .github/workflows	Jenkinsfile viết bằng Groovy	YAML .gitlab-ci.yml
Runner	Của GitHub hoặc tự host	Toàn bộ do bạn quản lý	Shared runner hoặc tự host
Hệ sinh thái	Marketplace hàng nghìn action	Kho plugin lớn và lâu năm	Tính năng tích hợp sẵn
Chi phí	Miễn phí cho repo công khai	Miễn phí, trả bằng vận hành	Có hạn mức phút miễn phí
Phù hợp với	Dự án đã dùng GitHub	Hạ tầng nội bộ, yêu cầu riêng	Team dùng GitLab toàn diện

Buổi học dùng GitHub Actions: mô hình sự kiện – job – step giống cả ba công cụ, nên nắm được một cái thì chuyển sang cái khác chỉ là đổi cú pháp.

© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

Kiến trúc GitHub Actions

Bốn cấp lồng nhau: một workflow chứa nhiều job, một job chứa nhiều step

EVENT · push / pull_request

WORKFLOW · .github/workflows/ci.yml

JOB · build (runs-on: ubuntu-latest)

- STEP · uses: actions/checkout@v4
- STEP · uses: actions/setup-java@v4
- STEP · run: ./gradlew test

JOB · deploy (needs: build)

- STEP · uses: appleboy/ssh-action@v1

Workflow
Một file YAML trong .github/workflows, mô tả toàn bộ pipeline cho một mục đích

Job
Một nhóm công việc chạy trên cùng một máy; các job mặc định chạy song song

Step
Một thao tác trong job: uses gọi action có sẵn, run thực thi lệnh shell

Action
Đoạn logic đóng gói tái dùng được, lấy từ Marketplace hoặc tự viết

Runner
Máy ảo sạch thực thi job, bị xoá ngay sau khi job kết thúc

Job chạy song song trừ khi khai báo needs; mỗi job có một runner riêng, không chia sẻ ổ đĩa.

© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

RIKKEI Academy®
where the dream begins

2 Thiết kế quy trình CI tự động

Sự kiện khởi động pipeline

Khối on: quyết định pipeline chạy khi nào — chọn sai sự kiện là chạy thừa hoặc không chạy

Sự kiện	Cú pháp	Mục đích sử dụng
Push lên nhánh	on: push: branches: [main]	Chạy CI cho nhánh chính sau mỗi lần merge
Pull Request	on: pull_request	Kiểm tra trước khi merge — bắt buộc phải có
Gắn tag phiên bản	on: push: tags: ['v*']	Kích hoạt quy trình phát hành chính thức
Theo lịch định kỳ	on: schedule: cron: '0 2 * * *'	Quét bảo mật, dọn cache, kiểm thử hồi quy
Chạy tay	on: workflow_dispatch	Triển khai lại hoặc rollback khi cần
Gọi từ workflow khác	on: workflow_call	Tái sử dụng một pipeline dùng chung cho nhiều repo

Cấu hình tối thiểu cho một dự án: push vào main để bảo vệ nhánh chính, cộng pull_request để không có thay đổi nào vào main mà chưa qua kiểm thử.

© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

Bốn giai đoạn của quy trình CI

Thứ tự được xếp theo nguyên tắc thất bại sớm: bước nhanh và rẻ chạy trước

1

10 giây

Checkout

Tải mã nguồn về runner

2

25 giây

Lint

Kiểm tra cú pháp và quy ước

3

1 phút 40

Unit Test

Chạy toàn bộ test, đo coverage

4

55 giây

Build

Đóng gói .jar và lưu artifact

Thất bại sớm, phản hồi nhanh

Lint sai thì dừng ngay, không tốn hai phút chạy test rồi mới báo lỗi

Mỗi bước một log riêng biệt

Pipeline đỏ ở đâu thì mở đúng bước đó, không phải đọc toàn bộ output

Là điều kiện bắt buộc để merge

Bật branch protection thì pull request đỏ sẽ không thể merge được

Tổng thời gian mục tiêu cho một lần chạy CI: dưới 5 phút. Lâu hơn thì lập trình viên sẽ ngừng chờ kết quả.

© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

Kịch bản workflow CI

Toàn bộ bốn giai đoạn trên, viết thành một file YAML duy nhất

.github/workflows/ci.yml

```yaml
name: CI
on:
  push: { branches: [ main ] }
  pull_request:

jobs:
  build:
    runs-on: ubuntu-latest
    timeout-minutes: 10
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-java@v4
        with: { java-version: '21', cache: gradle }
      - name: Lint
        run: ./gradlew -Dorg.gradle.daemon=false check
      - name: Unit test
        run: ./gradlew -Dorg.gradle.daemon=false test
      - name: Build
        run: ./gradlew -Dorg.gradle.daemon=false build -x test
```

1. **on: push và pull_request**
   Hai sự kiện tối thiểu: bảo vệ nhánh main và kiểm tra trước khi merge

2. **runs-on: ubuntu-latest**
   Máy ảo sạch do GitHub cấp cho mỗi lần chạy và xoá ngay sau khi kết thúc

3. **cache: gradle**
   Tái sử dụng thư mục ~/.gradle giữa các lần chạy, giảm phần lớn thời gian tải dependency

4. **timeout-minutes**
   Chặn job treo vô hạn; luôn đặt giới hạn cho mọi job trong pipeline

© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

Tối ưu thời gian chạy CI

Cùng một bộ test, thời gian chờ giảm gần bốn lần nhờ bốn thay đổi cấu hình

TRƯỚC TỐI ƯU

6 ph 40

Tải lại toàn bộ dependency mỗi lần chạy

SAU TỐI ƯU

1 ph 50

Cache dependency và chạy song song bằng matrix

Kỹ thuật	Cách làm	Hiệu quả
Cache dependency	setup-java với cache: gradle	**Bớt khoảng 2 phút mỗi lần chạy**
Chạy song song bằng matrix	strategy.matrix cho nhiều phiên bản	**Tổng thời gian bằng job dài nhất**
Lọc theo đường dẫn	on.push.paths-ignore: ['docs/**']	**Không chạy pipeline khi chỉ sửa tài liệu**
Timeout và fail-fast	timeout-minutes, fail-fast: true	**Dừng ngay khi đã biết chắc thất bại**

Kiểm tra hiệu quả cache bằng dòng "Cache restored from key" ở đầu log job; không thấy dòng đó nghĩa là cache chưa hoạt động.

© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

RIKEI Academy®
where the dream begins

3

Thiết kế quy trình CD tự động

Các chiến lược triển khai

Ba cách đưa phiên bản mới lên server, khác nhau ở thời gian gián đoạn và chi phí tài nguyên

Recreate

Xoá container cũ rồi tạo container mới cùng tên

2–10 giây

gián đoạn

Hành vi mặc định của docker compose up, phù hợp staging

Rolling update

Thay lần lượt từng instance, luôn còn instance phục vụ

0 giây

gián đoạn

Cần nhiều instance và một orchestrator như Swarm hoặc Kubernetes

Blue-Green

Dựng song song bản mới rồi chuyển toàn bộ traffic sang

0 giây

gián đoạn

Chỉ cần reverse proxy; tốn gấp đôi tài nguyên trong lúc chuyển

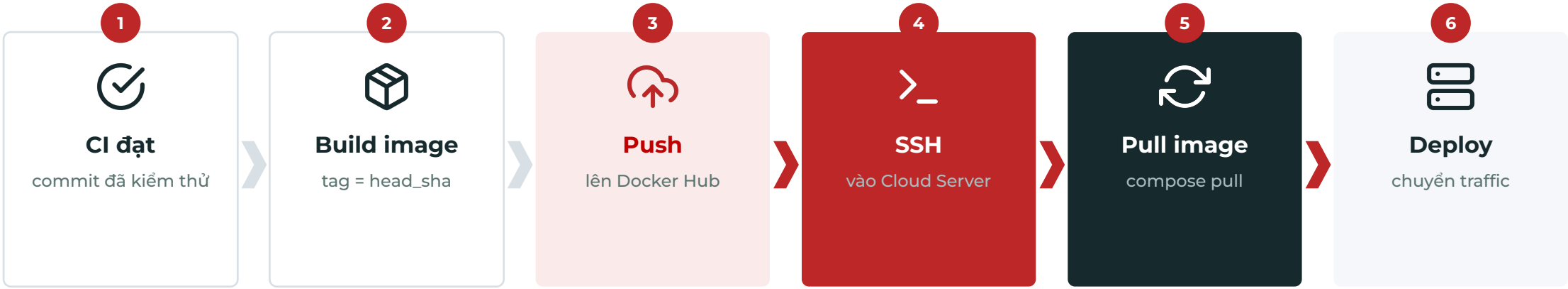
Lộ trình trong buổi học: đối với môn DevOps Fundamentals, phần 3 triển khai theo Recreate bằng docker compose up để hiểu cơ chế. Các kỹ thuật Rolling Update, Blue-Green sẽ học trong các môn học DevOps nâng cao.

Chọn chiến lược trước khi viết pipeline: mỗi chiến lược đòi hỏi cấu hình hạ tầng khác nhau, không phải chỉ đổi vài dòng YAML.

© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

Luồng CD từ artifact tới server

Sáu chặng nối tiếp; mỗi chặng chỉ chạy khi chặng trước kết thúc thành công




```
$ docker build -t $IMG:$SHA .   # SHA = head_sha của CI
$ docker push $IMG:$SHA
$ ssh deploy@server 'TAG=$SHA /srv/app/deploy.sh'
```

Tag bằng head_sha của CI
Image ứng với đúng commit vừa qua kiểm thử, không phải commit mới nhất

Không build lại trên server
Server chỉ kéo image đã kiểm thử, không cài JDK hay Maven

Rollback là đổi tag
Chạy lại deploy.sh với TAG cũ là về được phiên bản trước

© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

Kịch bản workflow CD

Kích hoạt khi CI hoàn tất, kiểm tra kết quả rồi mới triển khai; mọi khoá lấy từ Secrets

.github/workflows/cd.yml

```yaml
name: CD
on:
  workflow_run: { workflows: [CI], types: [completed] }
jobs:
  deploy:
    if: ${{ github.event.workflow_run.conclusion == 'success' }}
    runs-on: ubuntu-latest
    environment: production
    env: { SHA: '${{ github.event.workflow_run.head_sha' }} }
    steps:
      - uses: actions/checkout@v4
        with: { ref: '${{ env.SHA }}' }   # đúng commit đã qua CI
      - uses: docker/login-action@v3
        with:
          username: ${{ secrets.DOCKERHUB_USER }}
          password: ${{ secrets.DOCKERHUB_TOKEN }}
      - run: docker build -t $IMG:$SHA . && docker push $IMG:$SHA
      - uses: appleboy/ssh-action@v1
        with:
          host:   ${{ secrets.SSH_HOST }}
          key:    ${{ secrets.SSH_PRIVATE_KEY }}
          script: TAG=$SHA /srv/app/deploy.sh
```

1. **workflow_run cộng điều kiện if**

   workflow_run kích hoạt CD khi CI hoàn tất, kể cả khi thất bại; chỉ điều kiện if mới lọc ra lần chạy thành công

2. **head_sha thay cho github.sha**

   Trong ngữ cảnh workflow_run, github.sha trỏ tới nhánh mặc định chứ không phải commit vừa qua CI

3. **environment: production**

   Gắn job vào một môi trường có secrets riêng và có thể bắt buộc phê duyệt trước khi chạy

4. **script gọi deploy.sh**

   Toàn bộ logic triển khai nằm trong script trên server; pipeline chỉ truyền tag image vào

© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

RIKKEI Academy®
where the dream begins

4

Viết file Pipeline hoàn chỉnh

PHẦN 4

04 Kịch bản Pipeline và Secrets

Ghép CI và CD thành một kịch bản hoàn chỉnh, và bảo vệ các thông tin nhạy cảm bằng Environment Secrets.

- **Environment Secrets**
- **Phạm vi truy cập**
- **Pipeline đầy đủ**
- **Xử lý sự cố**

© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

Quản lý Environment Secrets

Bốn cấp lưu trữ với phạm vi truy cập khác nhau — chọn cấp hẹp nhất mà pipeline vẫn chạy

Repository secret — Mọi workflow trong repository đều đọc được

→ Token dùng chung, mức nhạy cảm thấp

Environment secret — Chỉ job có khai báo environment tương ứng

→ SSH private key và mật khẩu production

Organization secret — Nhiều repository trong cùng tổ chức

→ Token registry dùng chung toàn tổ chức

Variables (không mã hoá) — Đọc được và in ra được trong log

→ Tên image, hostname, cấu hình không bí mật

OIDC token ngắn hạn — Cấp riêng cho từng lần chạy rồi hết hiệu lực

→ Thay thế khóa tĩnh khi tích hợp với cloud

CƠ CHẾ PHÊ DUYỆT CỦA ENVIRONMENT

1. **Job khai báo environment: production**
   GitHub nhận diện job cần môi trường được bảo vệ
2. **Pipeline tạm dừng tại chỗ**
   Thông báo được gửi tới danh sách reviewer đã cấu hình
3. **Reviewer phê duyệt hoặc từ chối**
   Quyết định được ghi lại kèm người thực hiện và thời điểm
4. **Job mới được cấp secrets và chạy**
   Trước khi phê duyệt, runner không hề nhận được khoá nào

Ba lớp bảo vệ khác của environment: required reviewers, wait timer và giới hạn nhánh được phép triển khai.

© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

Nguyên tắc bảo mật thông tin nhạy cảm

Pipeline là nơi tập trung nhiều khóa nhất trong dự án, nên cũng là mục tiêu đáng giá nhất

NÊN LÀM	KHÔNG LÀM
✓ Lưu mọi khoá bằng Environment Secrets, phân theo môi trường	✗ Viết khoá vào mã nguồn, file cấu hình hoặc lịch sử commit
✓ Dùng access token có thời hạn và phạm vi hẹp thay cho mật khẩu	✗ Dùng echo để in giá trị secret ra log khi gỡ lỗi pipeline
✓ Tạo SSH key riêng cho CI, chỉ cho phép đúng vài lệnh cần thiết	✗ Dùng chung một khoá cho cả staging và production
✓ Xoay vòng khoá định kỳ và ngay khi có người rời dự án	✗ Cho phép workflow chạy từ fork truy cập vào secrets
✓ Bật required reviewers cho environment production	✗ Lưu khoá trong variables vì giá trị đó đọc được trong log

Lưu ý về cơ chế che log: GitHub tự thay giá trị secret bằng dấu sao, nhưng chỉ khi chuỗi xuất hiện nguyên vẹn. Giá trị đã base64 hoặc bị cắt sẽ hiện nguyên văn.

© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

Kịch bản Pipeline hoàn chỉnh

Sáu job nối nhau bằng needs; hai job cuối gắn với hai môi trường khác nhau



Ba job đầu là CI, ba job sau là CD — tất cả nằm trong cùng một file YAML

khai báo phụ thuộc và điều kiện

```yaml
jobs:
  lint:    { runs-on: ubuntu-latest, ... }
  test:    { needs: lint,  ... }
  build:   { needs: test,  ... }
  image:   { needs: build, ... }
  staging: { needs: image, environment: staging }
  production:
    needs: staging
    environment: production     # cần phê duyệt
    if: startsWith(github.ref, 'refs/tags/v')
```

needs tạo thành đồ thị
Job không khai báo needs sẽ chạy song song; needs biến pipeline thành chuỗi có thứ tự

Một file hay hai file
Gộp một file khi pipeline ngắn; tách CI và CD khi hai quy trình có nhịp khác nhau

Điều kiện if lọc lần chạy
Ví dụ chỉ triển khai production khi commit được gắn tag phiên bản

© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

Xử lý sự cố pipeline thường gặp

Sáu tình huống xuất hiện gần như chắc chắn trong tháng đầu vận hành pipeline

Hiện tượng	Nguyên nhân thường gặp	Cách xử lý
Pipeline không được kích hoạt	Sai đường dẫn file hoặc sai tên nhánh trong khối on	Đối chiếu .github/workflows và branches
Test đạt ở máy, thất bại trên CI	Phụ thuộc thứ tự test, múi giờ hoặc dữ liệu cục bộ	Chạy test trong container giống CI
Thời gian build tăng dần	Không cache dependency nên tải lại mỗi lần	Bật cache và kiểm tra dòng cache hit
docker push bị từ chối	Access token hết hạn hoặc thiếu quyền ghi	Cấp lại token và cập nhật secret
Bước SSH deploy hết thời gian chờ	Firewall chặn IP runner hoặc sai host, sai port	Mở cổng cho runner, đối chiếu secrets
Deploy xong vẫn là bản cũ	Tag image không đổi nên server dùng lại cache	Tag theo git sha và thêm --pull always

Nguyên tắc gỡ lỗi: đọc log từ bước thất bại đầu tiên, không đọc từ dưới lên. Các lỗi phía sau thường chỉ là hệ quả của bước đầu tiên.

© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

TỔNG KẾT

- Phân biệt CI, Continuous Delivery và Continuous Deployment
- Thiết kế quy trình CI: lint, unit test và build sản phẩm
- Thiết kế quy trình CD: đóng gói image, triển khai sản phẩm
- Viết kịch bản YAML hoàn chỉnh và quản lý Secrets an toàn

© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

RIKEI Academy®
where the dream begins

KẾT THÚC

HỌC VIỆN ĐÀO TẠO LẬP TRÌNH CHẤT LƯỢNG NHẬT BẢN